

머신러닝 및 실습

(Lecture 12)

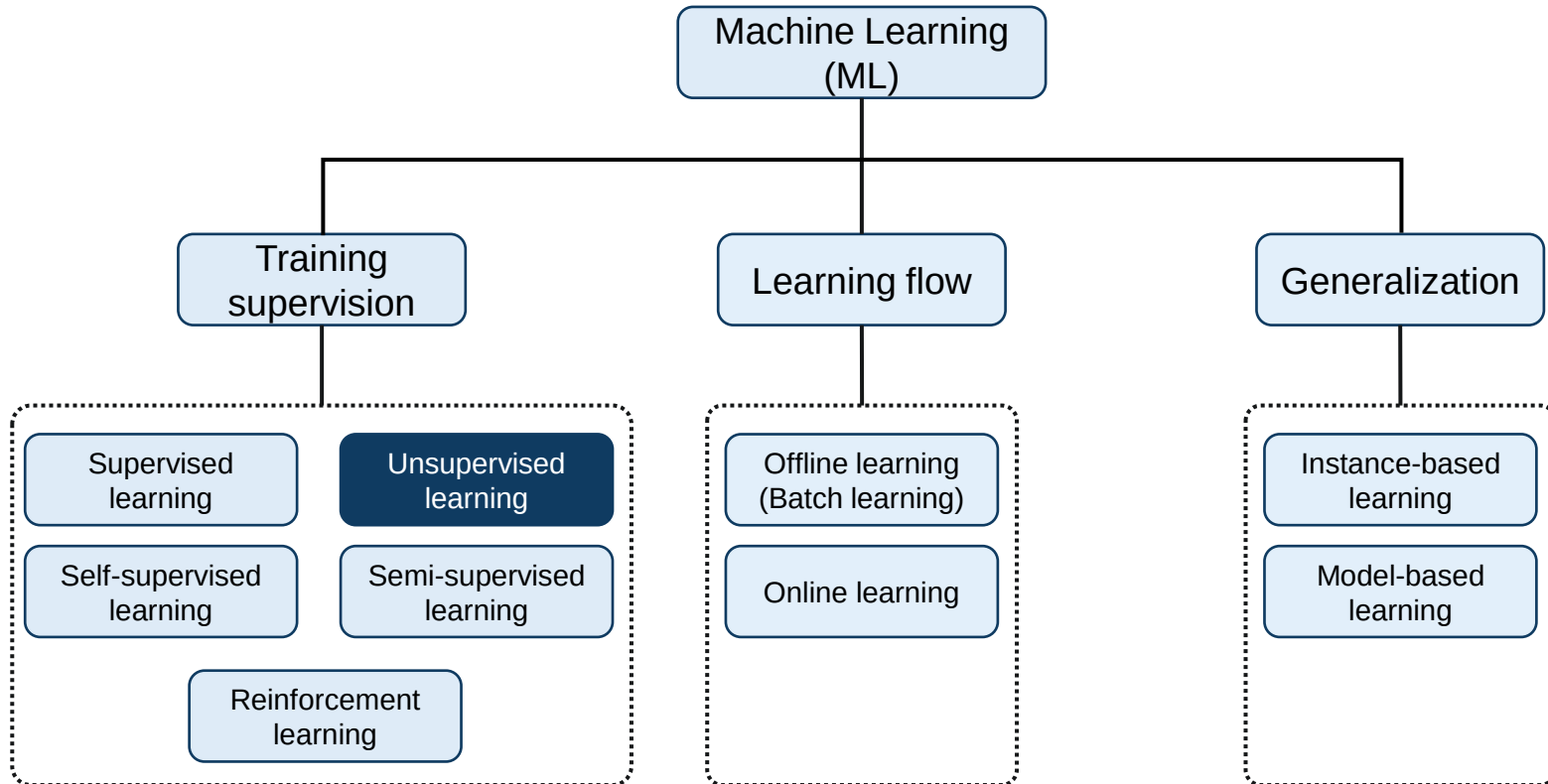
노연수

(esnoh@koreatech.ac.kr)

Course Plan

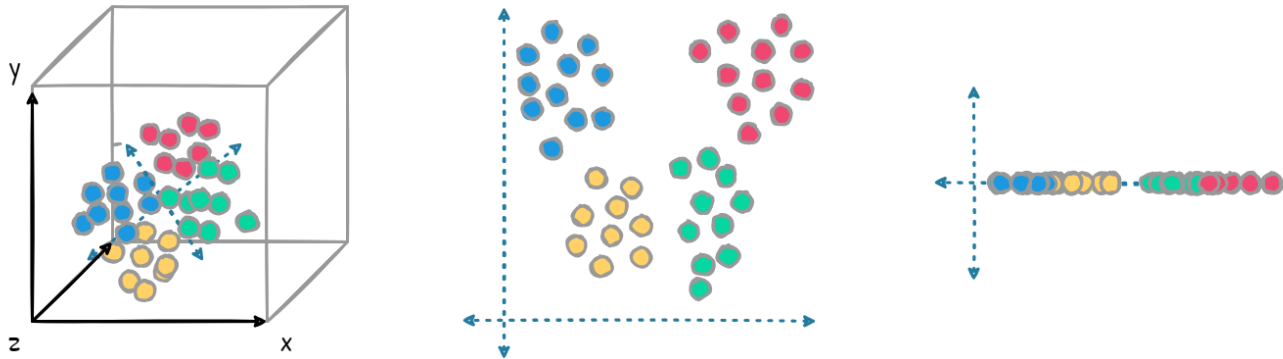
Week	Lecture	Contents	Notes
1	-	-	대체휴일
2	파이썬 기초 I	자료형, 연산자, 조건문, 반복문	
3	파이썬 기초 II	함수, 모듈, 패키지, 클래스	
4	머신러닝이란 무엇인가?	머신러닝 기초 개념, 머신러닝 시스템의 종류 등	
5	머신러닝의 전체적인 흐름 이해하기 I	프로젝트 방향 수립, 데이터 수집, 데이터 분석	
6	머신러닝의 전체적인 흐름 이해하기 II	데이터 전처리, 모델 학습, 모델 튜닝	
7	분류모델의 이해	이진 분류기 훈련, 성능 측정, 다중 분류	
8	중간고사	-	
9	회귀모델의 이해	선형 회귀, 경사 하강법	프로젝트 공지
10	회귀모델의 확장	다항 회귀, 로지스틱 회귀	
11	서포트 벡터 머신의 이해	규제 선형 모델, SVM 분류/회귀 등	
12	다양한 분류 모델과 앙상블	결정 트리, 랜덤 포레스트	
13	차원 축소	차원 축소, 주성분 분석(PCA)	이러닝(5/25~5/31)
14	군집 분석	K-means clustering, DBSCAN	
15	프로젝트 발표	프로젝트 발표, 보강(Week 1)	이러닝(6/8~6/14)
16	기말고사	기말고사(6/15, 09:00-11:00)	

■ 비지도 학습(Unsupervised learning)

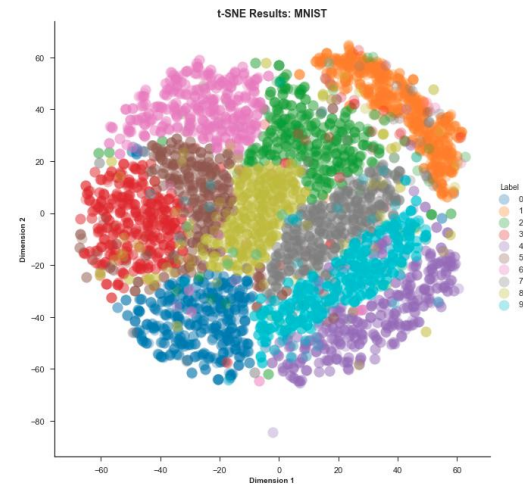


차원 축소(Dimensionality reduction)

▶ 데이터의 정보는 최대한 보존하면서 데이터의 크기와 특징의 개수를 줄이는 작업

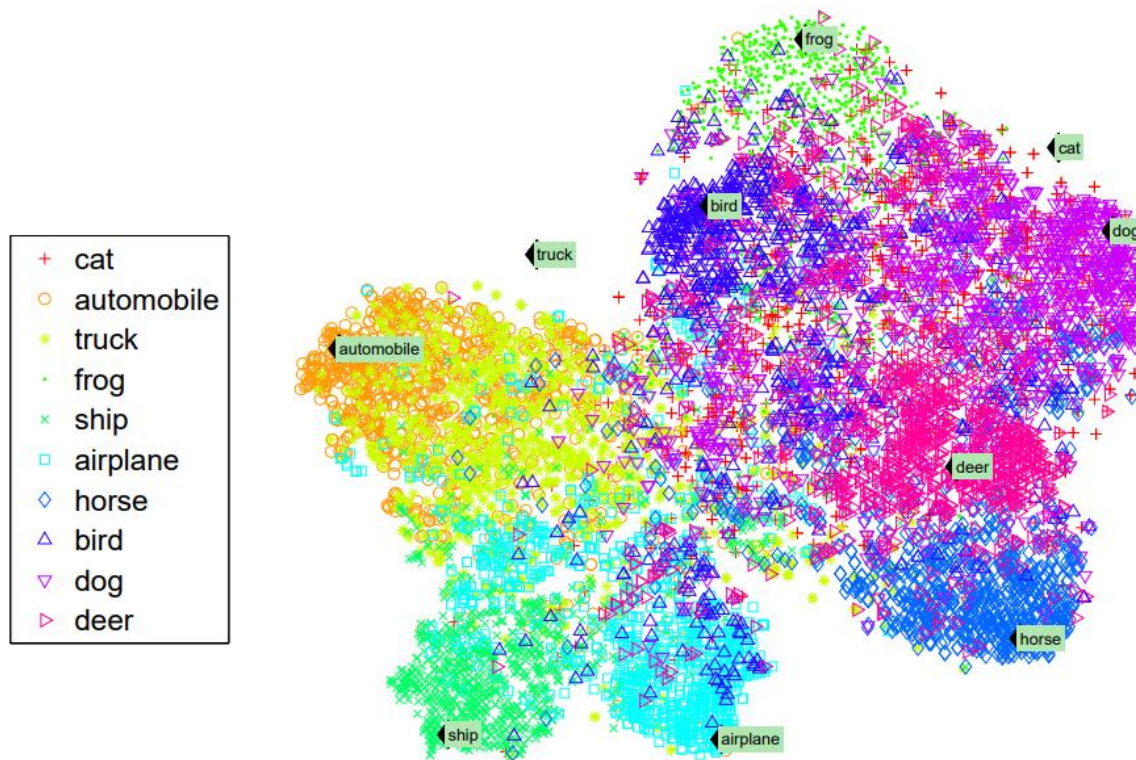


0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9 9 9 9 9 9



■ 군집(Clustering)

▶ 비슷한 샘플을 하나의 클러스터로 만드는 작업

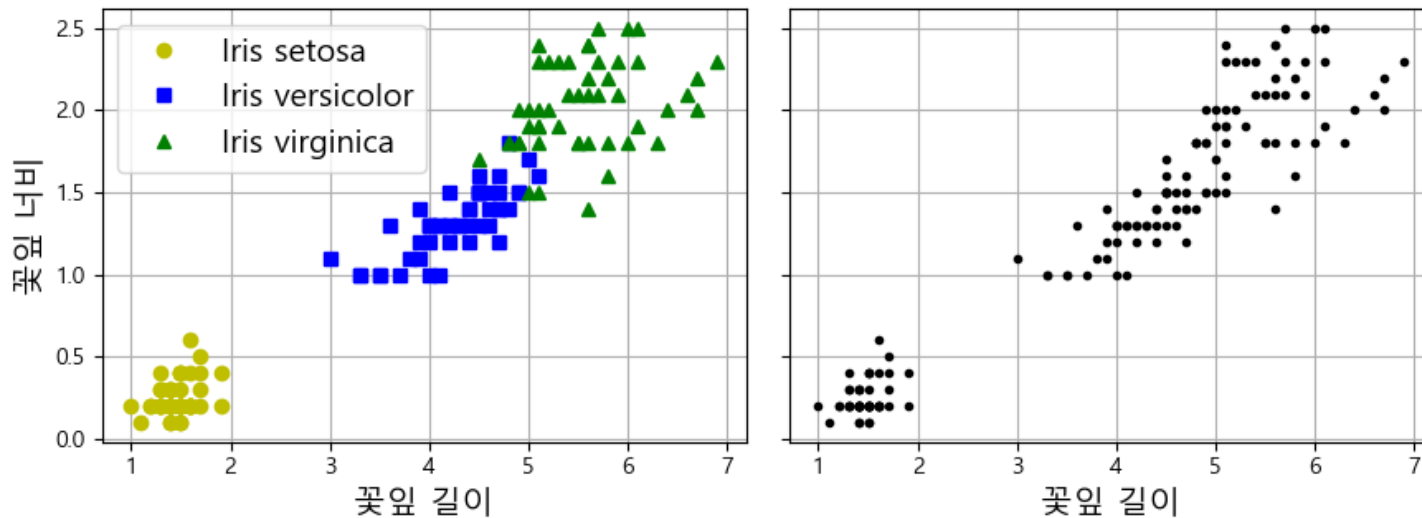


[Ref] Richard Socher et al., "Zero-Shot Learning Through Cross-Modal Transfer," *Proceedings of the 2th International Conference on Neural Information Processing Systems*, 1, pp.935-943, 2013

군집(Clustering)

분류(Classification)?

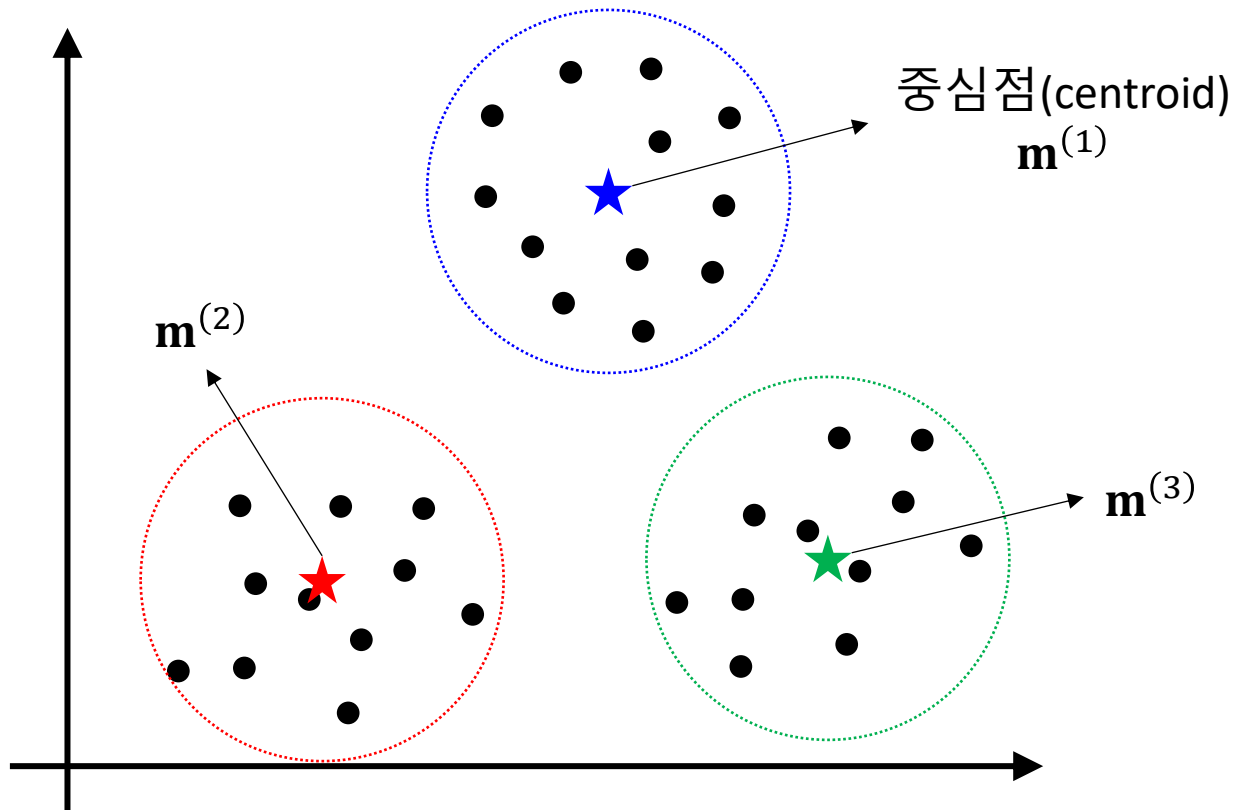
▶ 레이블이 없다면 지도 학습 기반의 분류 알고리즘 사용 불가



K-Means Clustering

K-평균 클러스터링

▶ 레이블이 없는 데이터를 유사한 특성을 가진 K개의 그룹으로 묶는 알고리즘

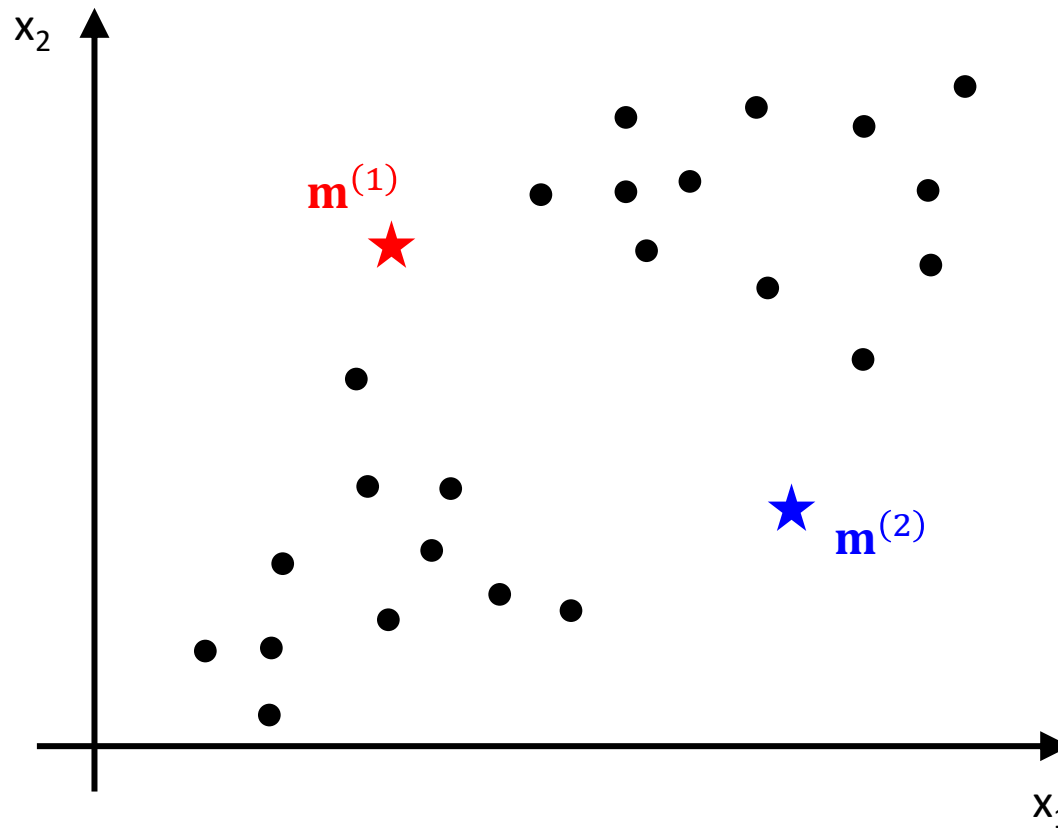


K-Means Clustering

알고리즘(1): Initialization

▶ K 클러스터 중심점들을 무작위로 초기화

- 중심점 $\mathbf{m}^{(k)} = (m_1^{(k)}, m_2^{(k)})$, $k = 1, \dots, K$

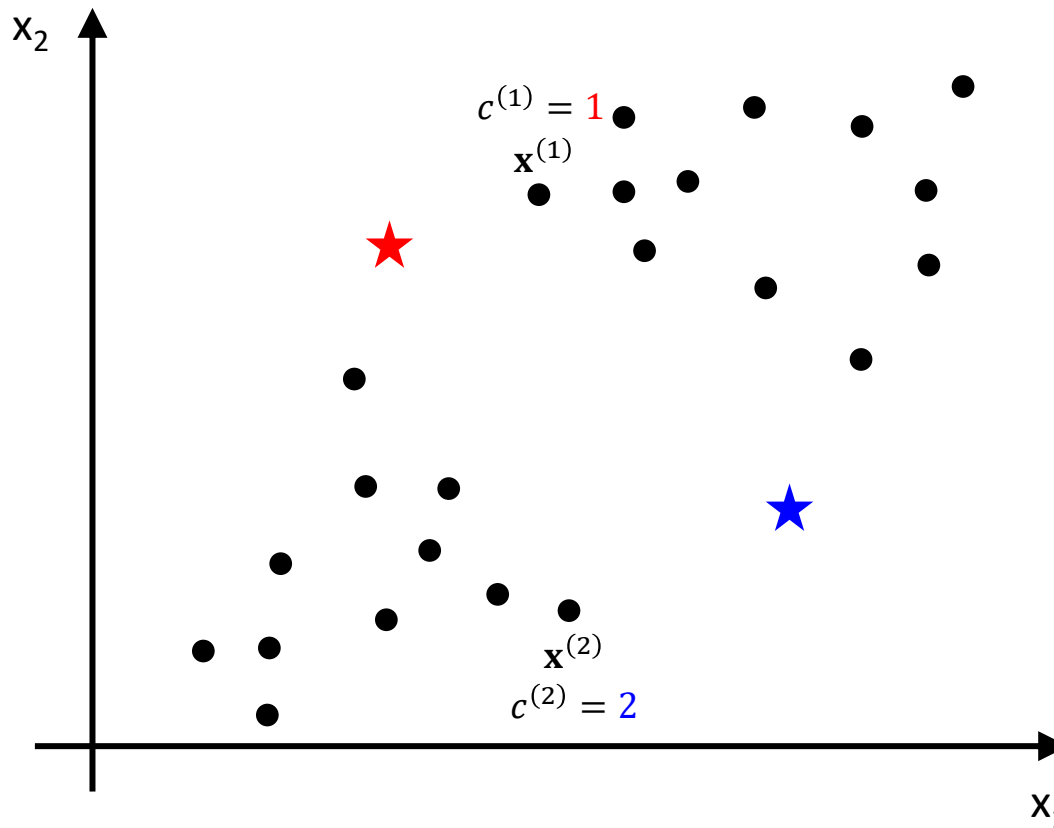


K-Means Clustering

알고리즘(2): Cluster assignment

▶ 각 샘플 $\mathbf{x}^{(i)} = (x_1^{(i)}, x_2^{(i)})$ 를 가까운 중심점의 클러스터로 할당

- $c^{(i)}$: $\mathbf{x}^{(i)}$ 에 할당되는 클러스터 인덱스 ($c^{(i)} = 1, \dots, K$)

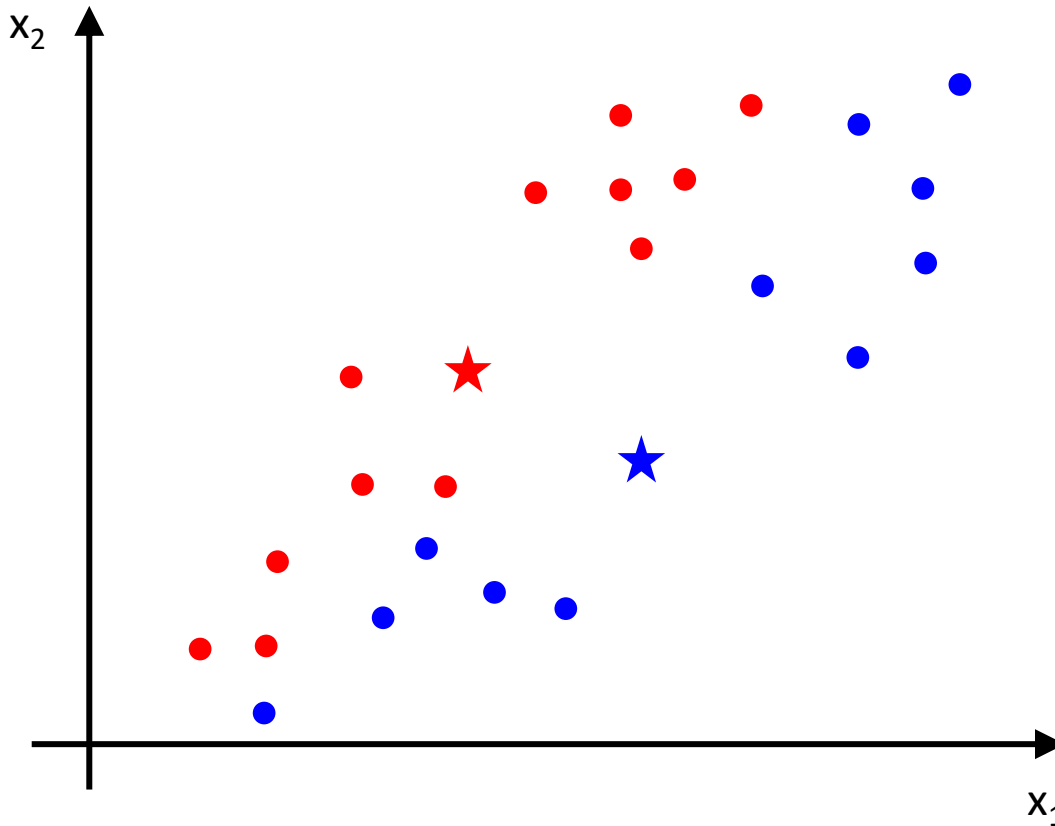


$$c^{(i)} = \underset{k}{\operatorname{argmin}} \|\mathbf{x}^{(i)} - \mathbf{m}^{(k)}\|^2$$
$$(i = 1, \dots, n) \quad (k = 1, \dots, K)$$

K-Means Clustering

알고리즘(3): Update cluster centroids

▶ 각 클러스터에 대한 새로운 중심점 계산



$$I_k^{(i)} = \begin{cases} 1, & \text{if } c^{(i)} = k \\ 0, & \text{if } c^{(i)} \neq k \end{cases}$$

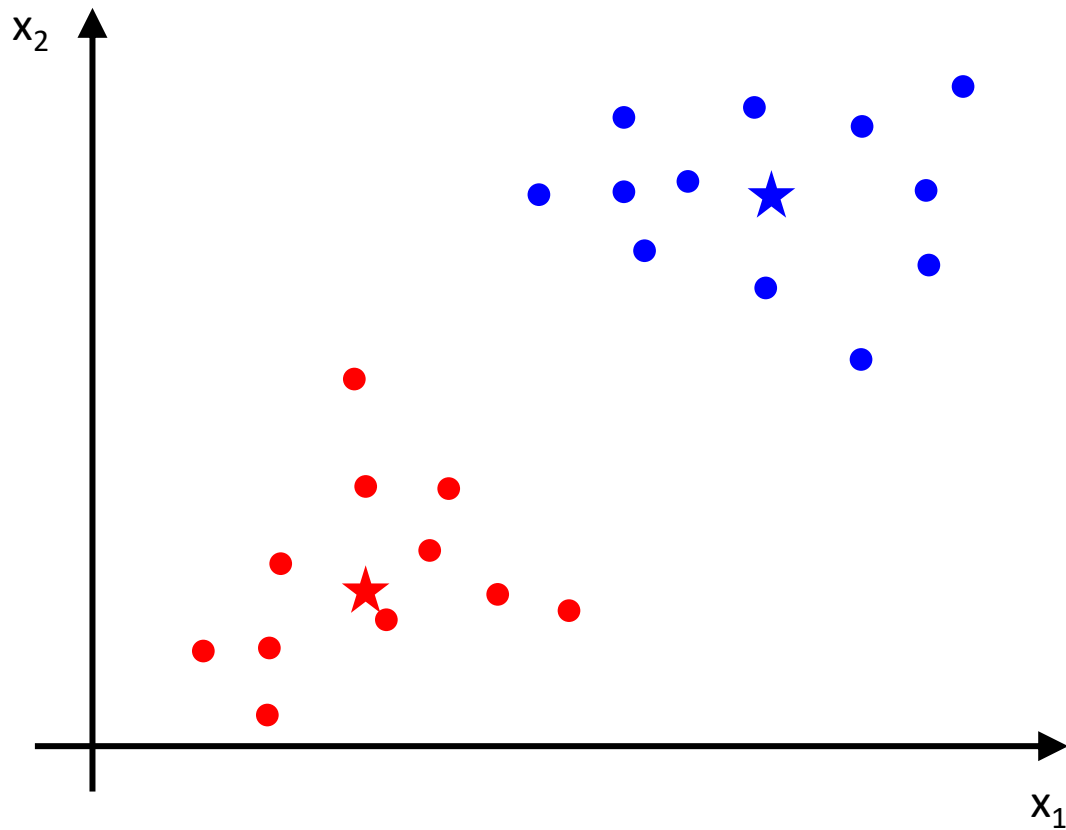
$$\mathbf{m}^{(k)} = \frac{\sum_i I_k^{(i)} \mathbf{x}^{(i)}}{\sum_i I_k^{(i)}}$$

K-Means Clustering

알고리즘(4): Repeat

▶ 클러스터 할당과 중심점 재계산 과정을 반복

- 중단 조건: 할당 시 클러스터가 바뀌는 샘플이 없을 때



비용 함수(Cost function): 이너셔(Inertia)

▶ 각 클러스터에 속한 샘플 $\mathbf{x}^{(i)}$ 들과 중심점 $\mathbf{m}^{(k)}$ 사이의 제곱 거리 합

$$J(\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(K)}) = \sum_{k=1}^K \sum_{\mathbf{x}^{(i)} \in S_k} \|\mathbf{x}^{(i)} - \mathbf{m}^{(k)}\|^2$$

클러스터 개수 $\uparrow$ K

$\mathbf{x}^{(i)} \in S_k$ $\downarrow$ k 번째 클러스터에 속한 샘플들의 집합

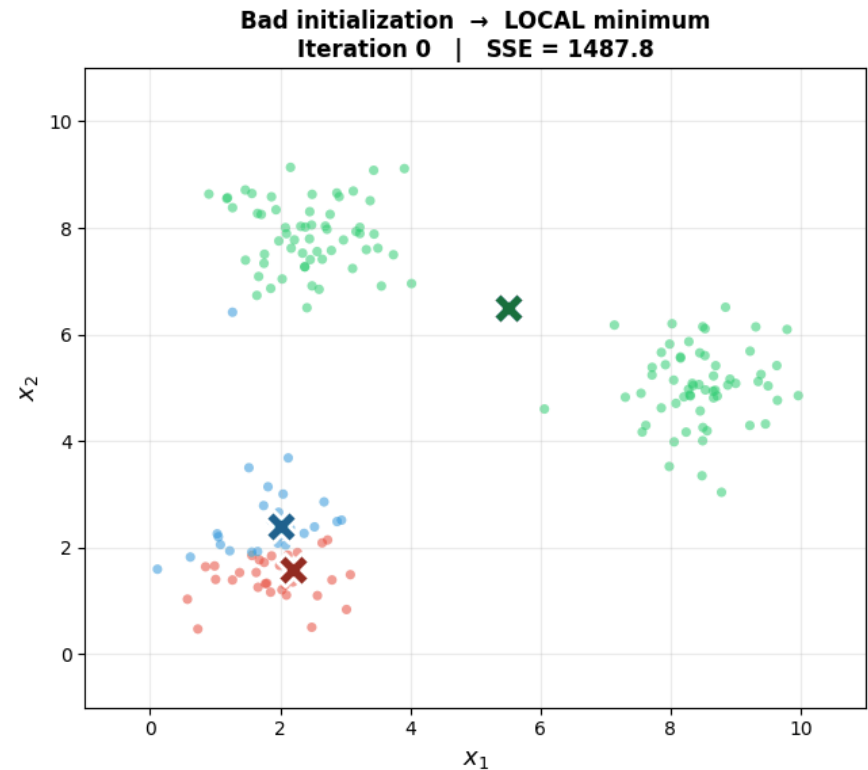
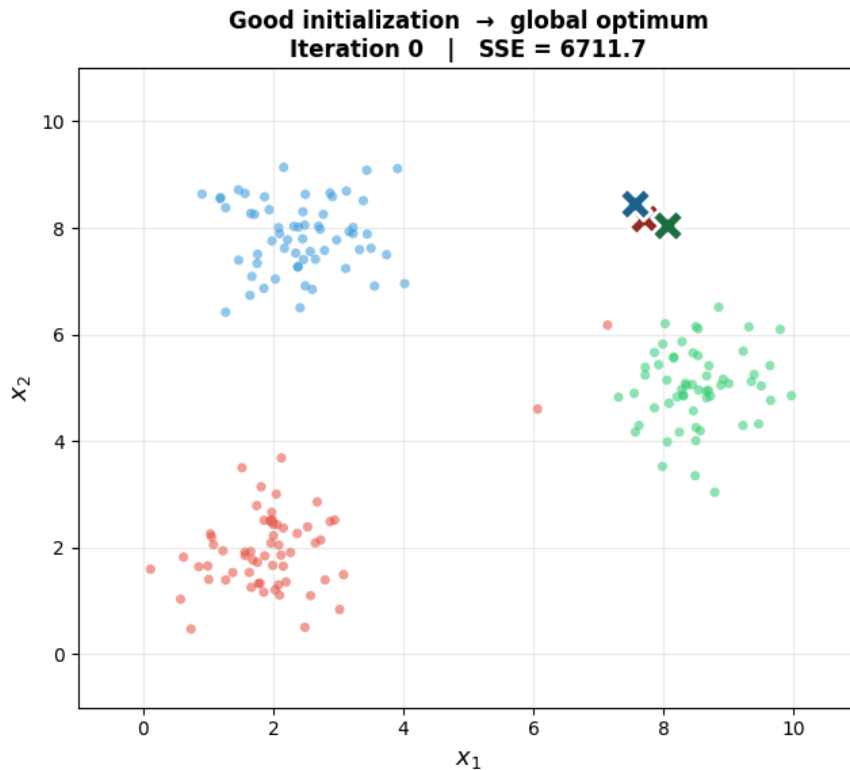
$\mathbf{m}^{(k)}$ $\uparrow$ j 번째 클러스터 중심점

$$\hat{\mathbf{m}}^{(1)}, \dots, \hat{\mathbf{m}}^{(K)} = \operatorname{argmin}_{\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(K)}} J(\mathbf{m}^{(1)}, \dots, \mathbf{m}^{(K)})$$

K-Means Clustering

중심점 초기화의 중요성

▶ 지역 최솟값(local minimum)에 빠질 우려가 있음



K-Means Clustering

■ 사이킷런 클래스: KMeans

▶ 매개변수

- `n_clusters`: 클러스터 개수(K)
- `init`: 중심점 리스트를 담은 넘파이 배열 혹은 'random' 입력
- `n_init`: 반복(iteration) 횟수

```
1 from sklearn.cluster import KMeans
2
3 good_init = np.array([[-3,3], [-3,2], [-3,1], [-1,2], [0,2]])
4 kmeans = KMeans(n_clusters=5, init=good_init, n_init=1, random_state=42)
5 kmeans.fit(X)
6
7 kmeans.inertia_
```

■ 사이킷런 클래스: KMeans

▶ 중심점 초기화 방법

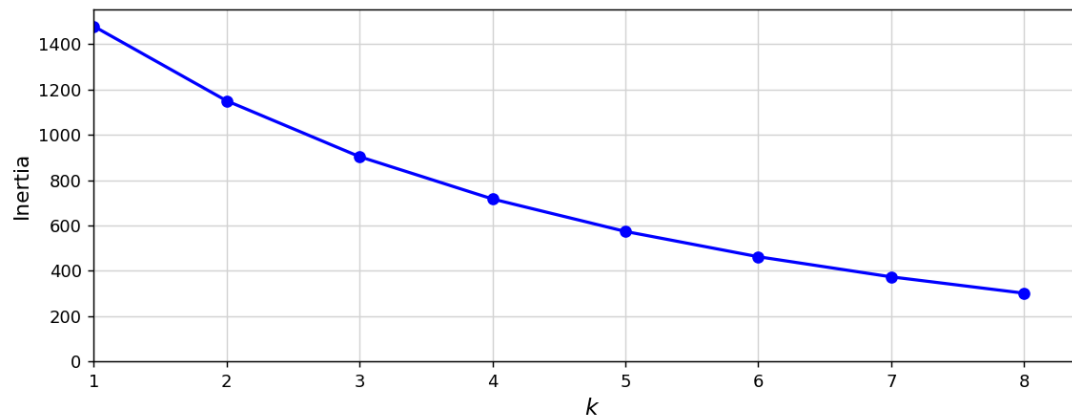
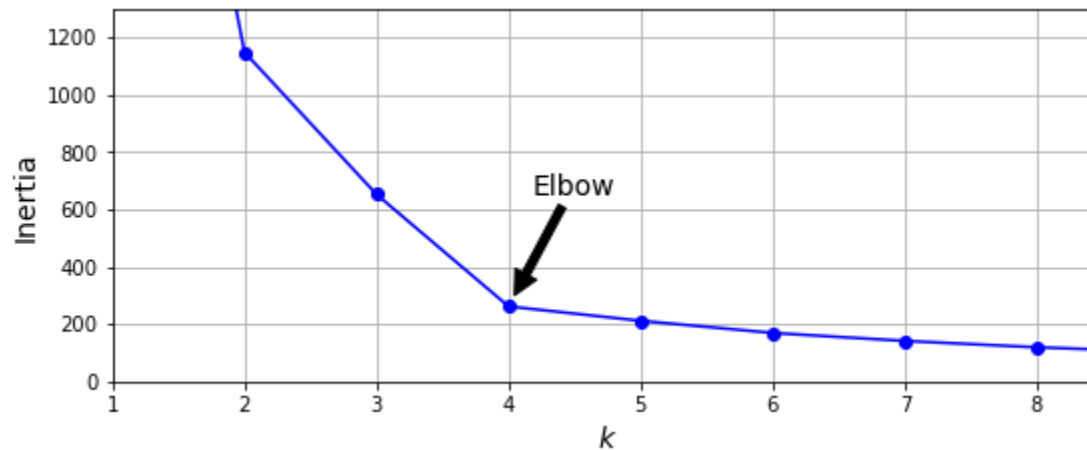
- 사용자가 정의한 횟수만큼 알고리즘 반복 수행
- 최종 이너셔(inertia)가 가장 낮은 모델을 반환 받아 사용

```
1 from sklearn.cluster import KMeans
2
3 kmeans = KMeans(n_clusters=5, n_init=10, random_state=42)
4 kmeans.fit(X)
5
6 kmeans.cluster_centers_
```

K-Means Clustering

■ 최적의 클러스터 개수 찾기(1)

▶ 클러스터 개수에 따른 최종 이너셔를 확인



❏ 최적의 클러스터 개수 찾기(2)

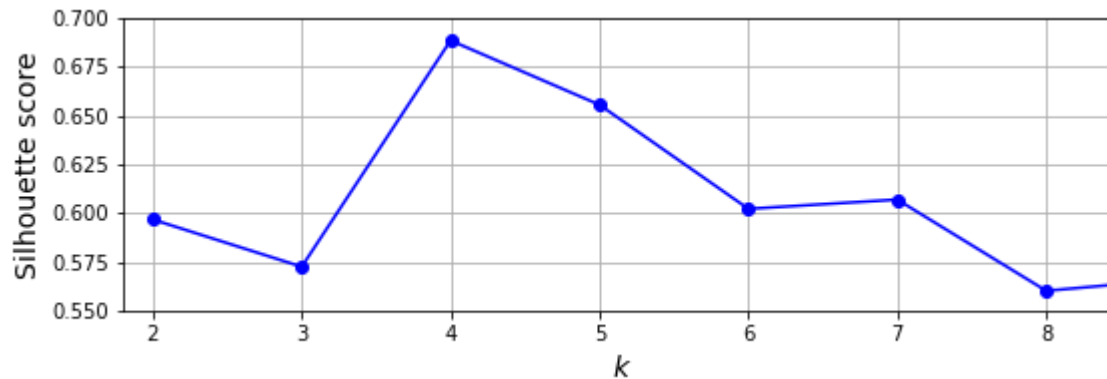
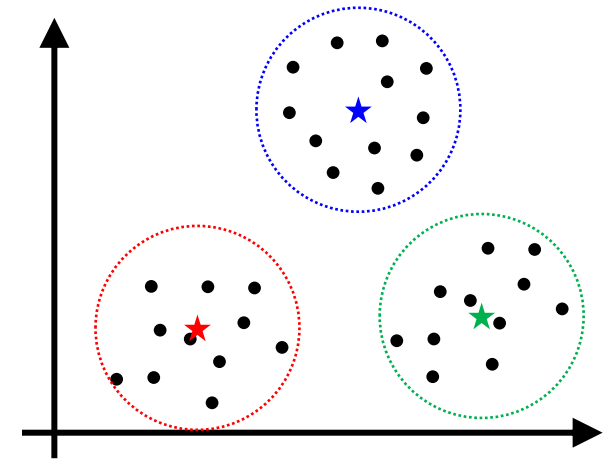
▶ 실루엣 점수(silhouette score) 확인

- 모든 샘플들에 대한 실루엣 계수(silhouette coefficient)의 평균

$$-1 \leq s(i) = \frac{(b - a)}{\max(a, b)} \leq 1$$

a : 클러스터 내부 평균 거리

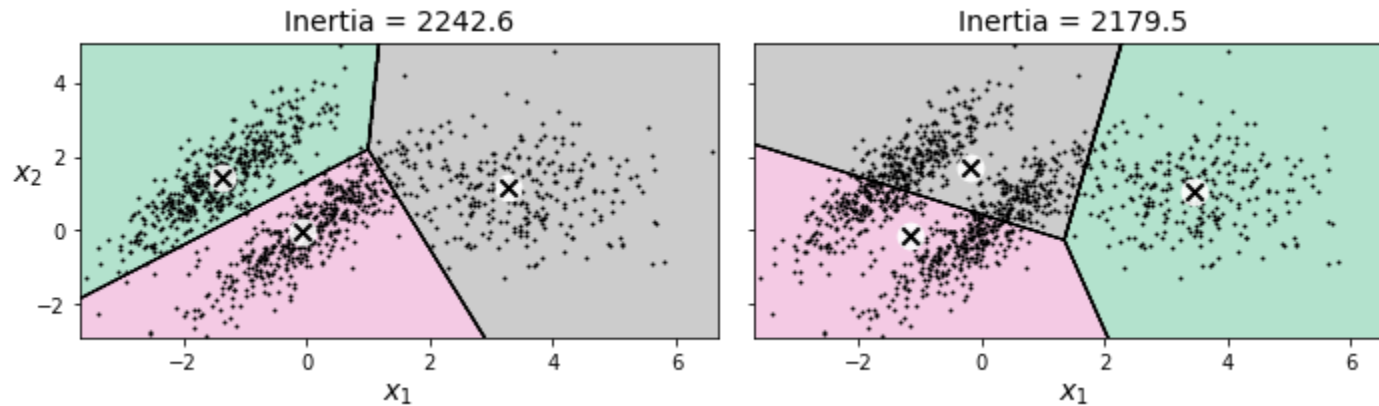
b : 가장 가까운 클러스터의 샘플까지 평균 거리



K-Means Clustering

한계

- ▶ 클러스터의 크기 혹은 밀집도가 다른 경우 잘 동작하지 않음



특징 스케일링(feature scaling)을 잘 하자

응용: 이미지 분할(image segmentation)

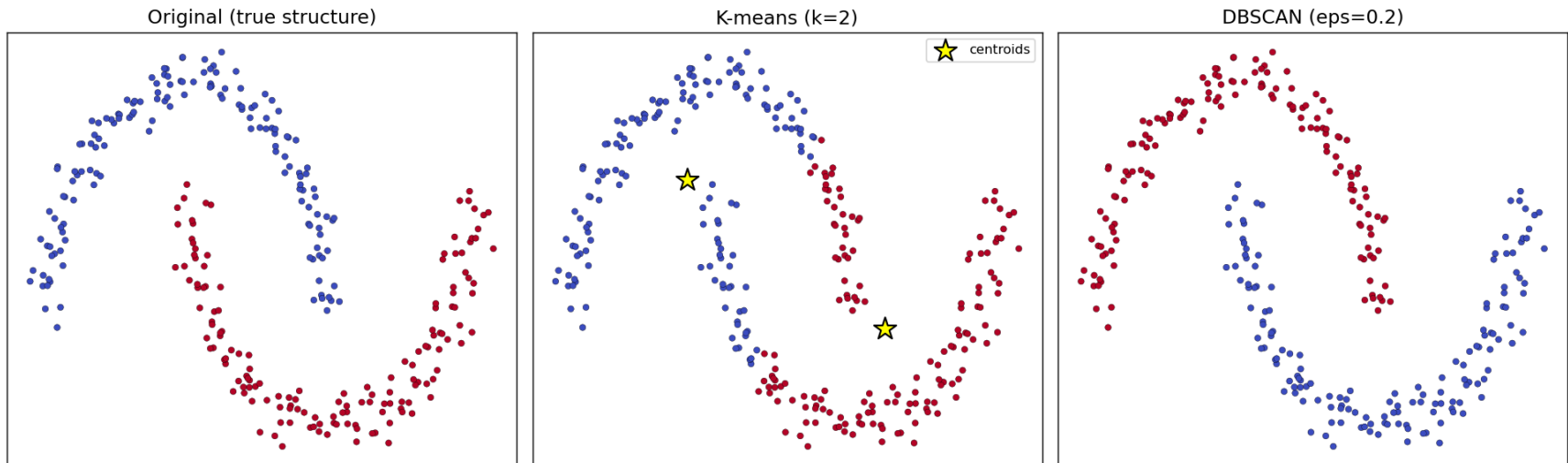
- ▶ 이미지를 여러 개의 세그먼트로 분할 – 색상 분할, 시맨틱 분할, 인스턴스 분할
 - 색상 분할: 동일한 색상을 가진 픽셀을 같은 세그먼트에 할당



■ Density-Based Spatial Clustering of Applications with Noise

▶ 데이터가 조밀한 영역을 클러스터로 묶는 밀도(density) 기반 군집 알고리즘

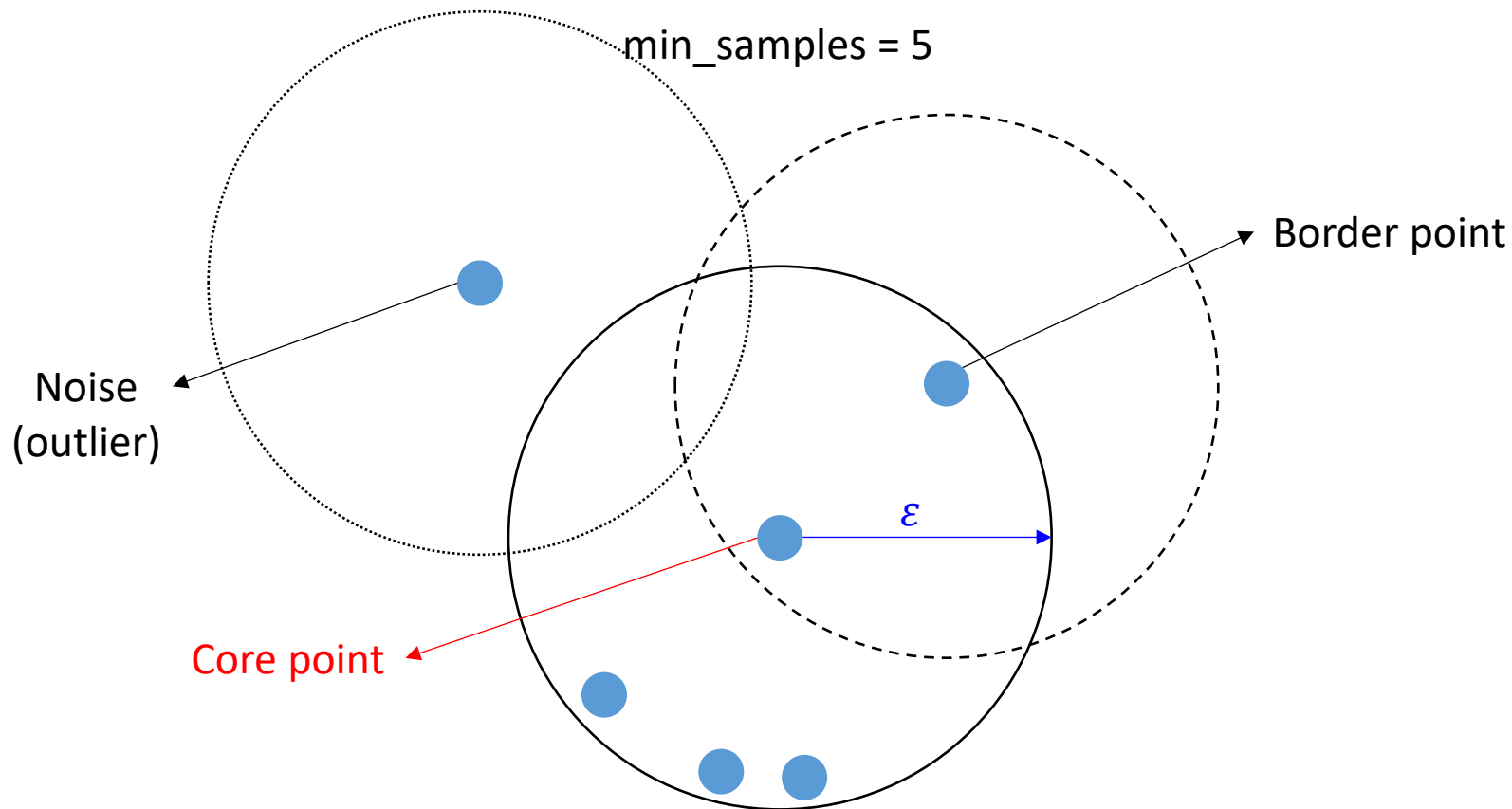
- 노이즈(Noise): 클러스터에 속하지 않는 이상치(outliers)



I 기본 개념

▶ 일정 반경 내에 데이터가 최소 개수 이상 존재 → **핵심 샘플(core point)**로 지정

- 반경(ϵ)
- 최소 샘플 수(min_samples)

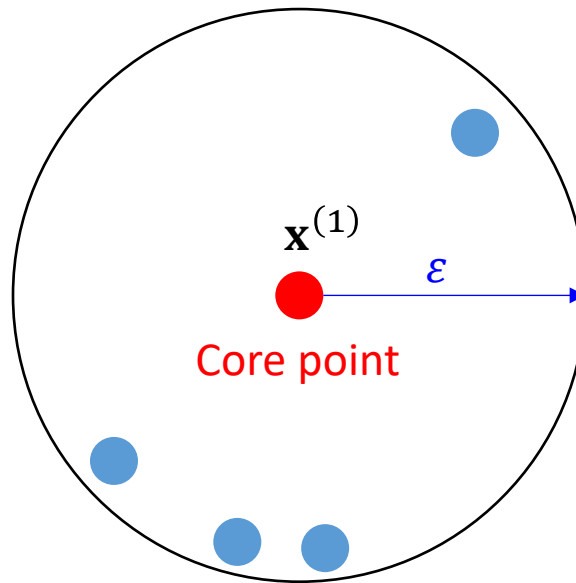


알고리즘 정의(1): ε -neighborhood of a point

▶ 두 샘플간 거리가 반경(ε) 이내인 점들의 집합

$$N_{eps}(\mathbf{x}^{(i)}) = \{ \mathbf{x}^{(i)} \in \mathbf{X} \mid \|\mathbf{x}^{(1)} - \mathbf{x}^{(i)}\| \leq \varepsilon \}$$

$$N_{eps}(\mathbf{x}^{(1)}) = 5$$

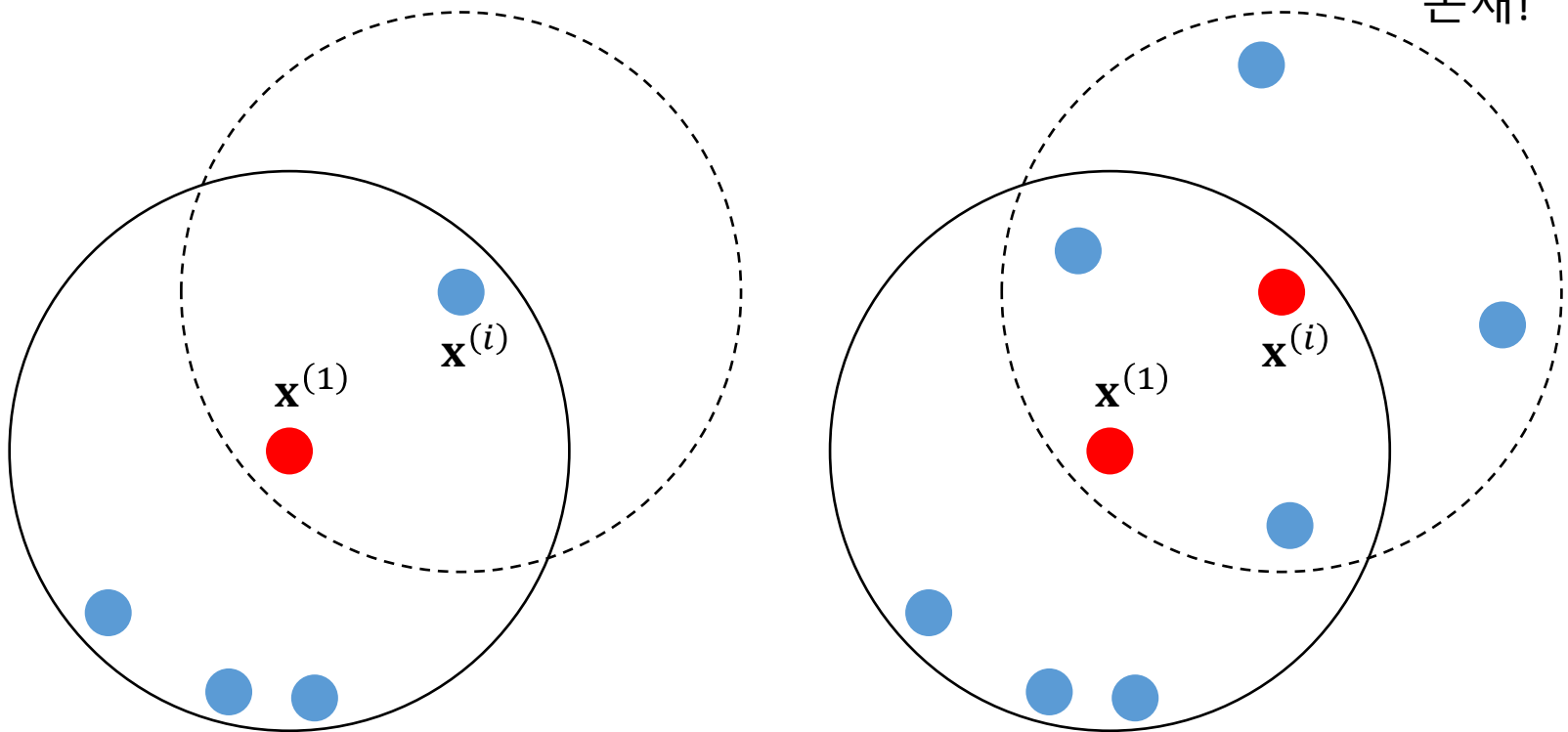


알고리즘 정의(2): Directly density-reachable

- ▶ i 번째 샘플 $\mathbf{x}^{(i)}$ 이 $N_{eps}(\mathbf{x}^{(1)})$ 에 속함
- ▶ $N_{eps}(\mathbf{x}^{(1)}) \geq \text{min_samples}$

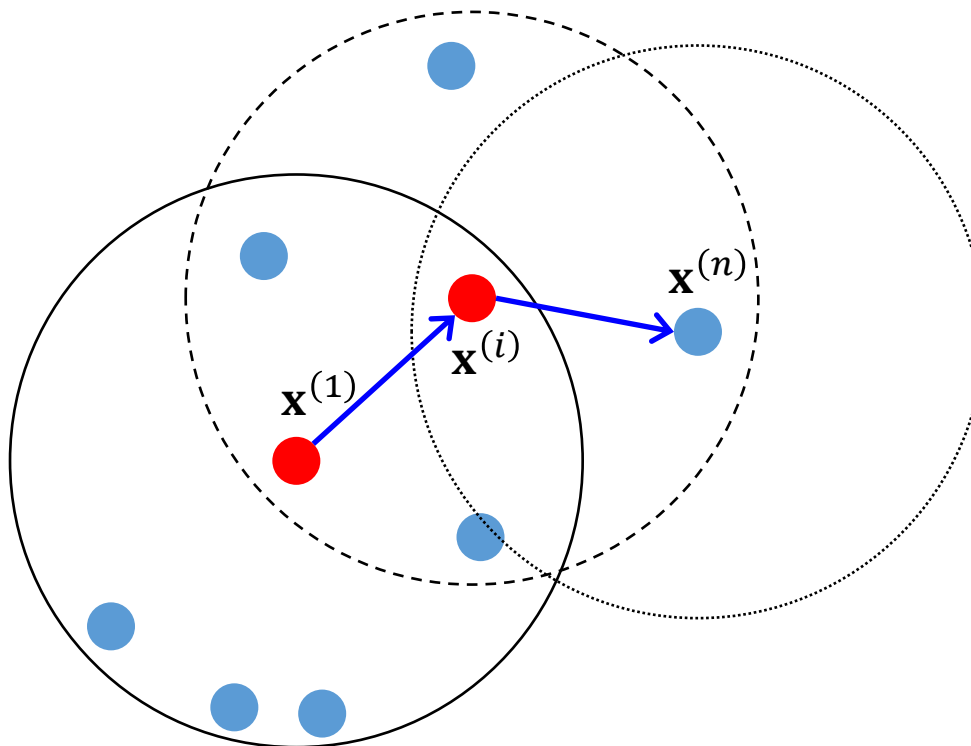
$\mathbf{x}^{(i)}$ Directly density-reachable from $\mathbf{x}^{(1)}$

대칭성(symmetric)
존재!



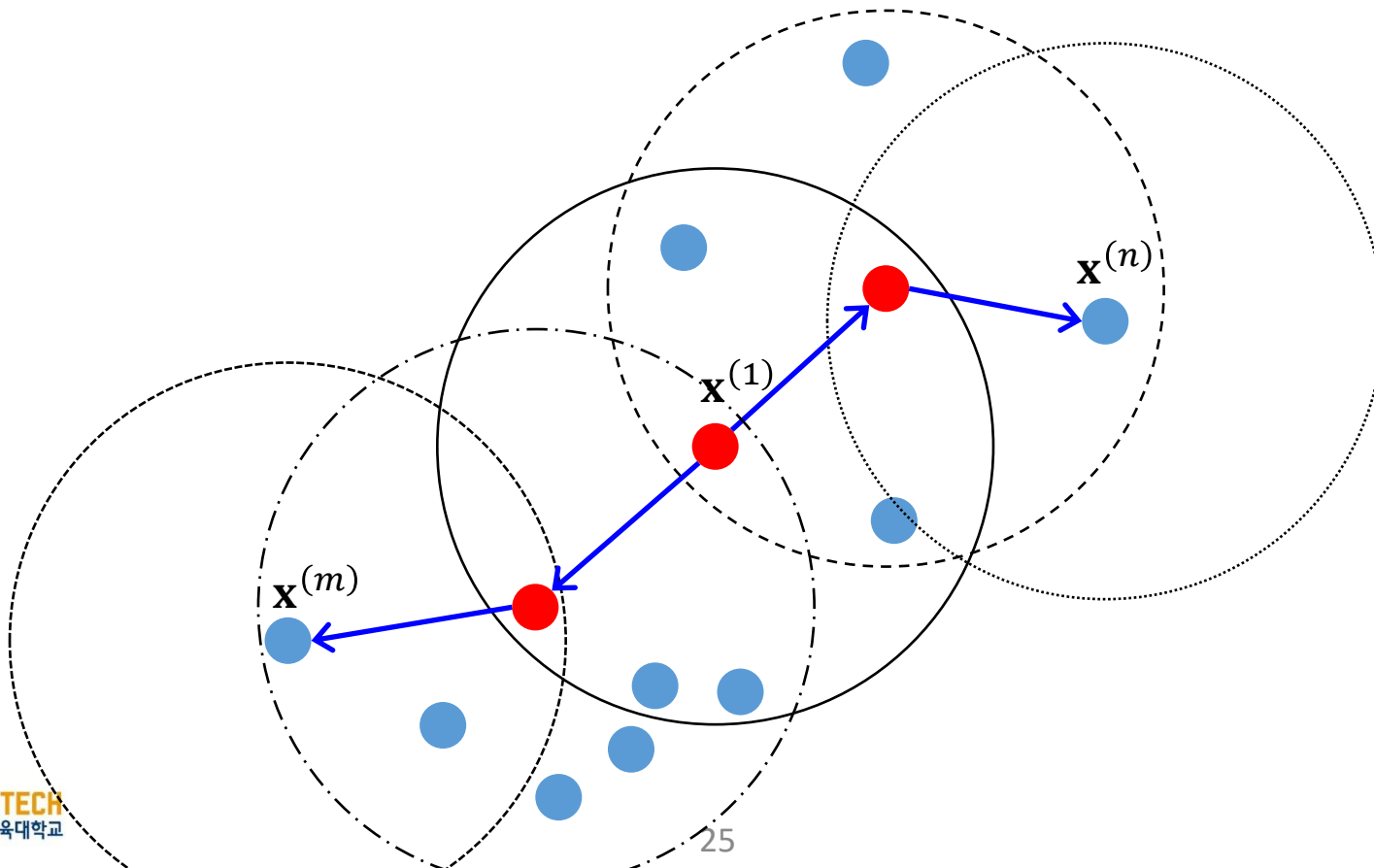
알고리즘 정의(3): Density-reachable

$\mathbf{x}^{(n)}$ Density-reachable from $\mathbf{x}^{(1)}$



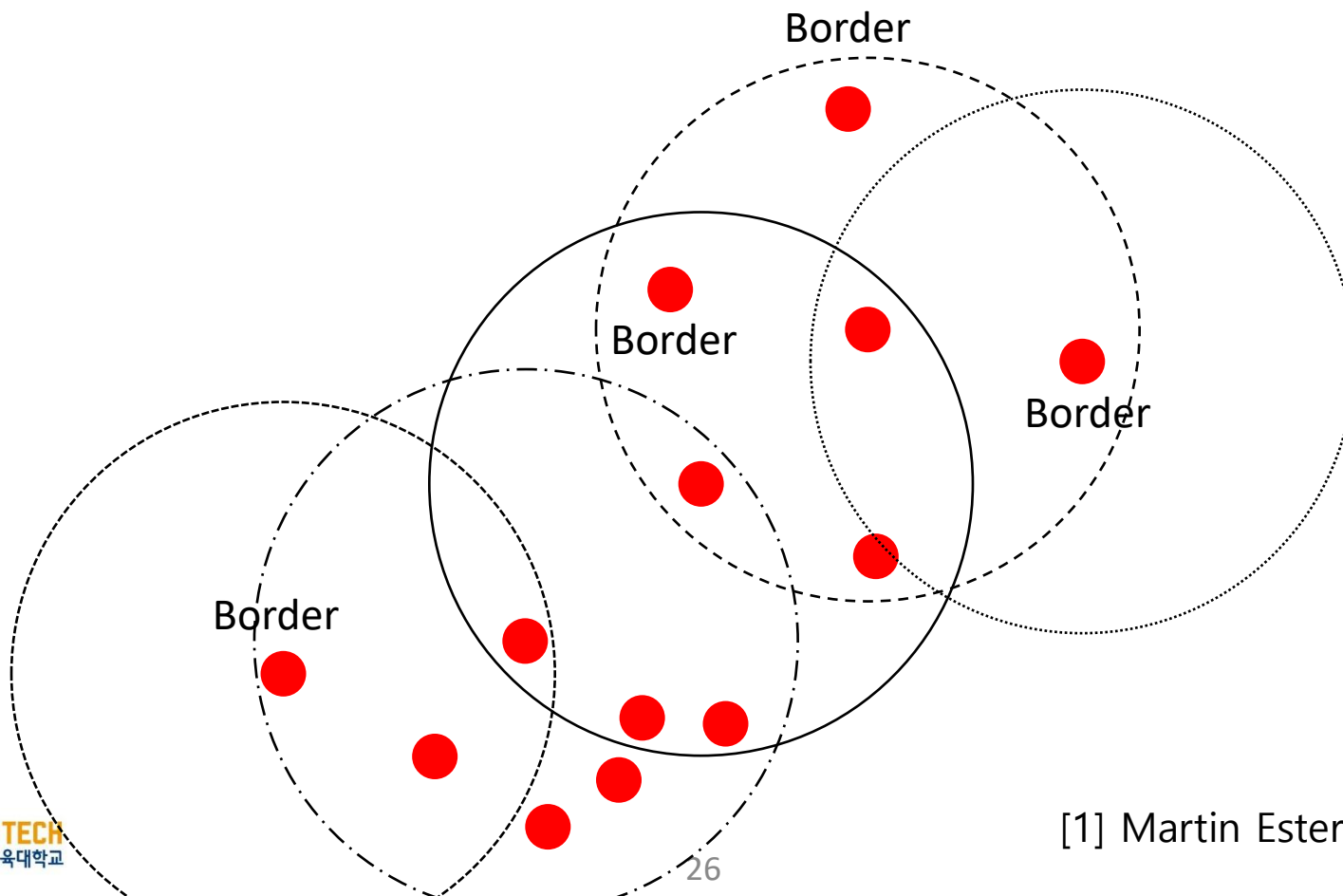
알고리즘 정의(4): Density-connected

$\mathbf{x}^{(n)}, \mathbf{x}^{(m)}$ **Density-connected** each other by $\mathbf{x}^{(1)}$



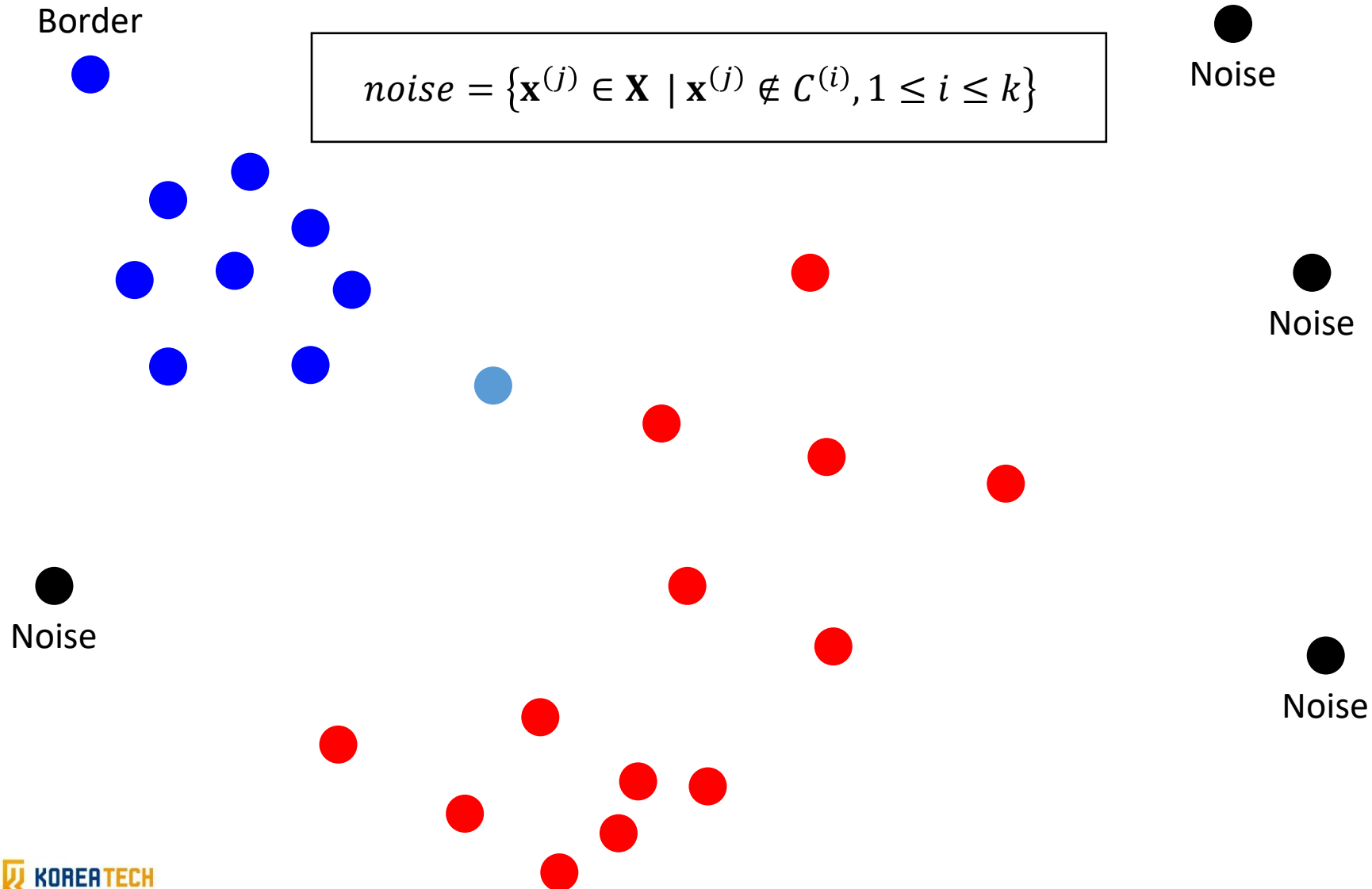
알고리즘 정의(5): 클러스터(Cluster)

"A set of **density-connected** points which is **maximal** with respect to **density-reachability**" [1]



[1] Martin Ester *et al.* (1996)

알고리즘 정의(6): 노이즈(Noise)



■ 사이킷런 클래스: DBSCAN

▶ 매개변수

- eps: 반경
- min_samples: 핵심 포인트 선정을 위한 최소 샘플 갯수

```
1 from sklearn.cluster import DBSCAN
2
3 dbscan = DBSCAN(eps=0.05, min_samples=5)
4 dbscan.fit(X)
5
6 # 샘플이 속한 클러스터(-1: 이상치)
7 dbscan.labels_
8
9 # 핵심 포인트 인덱스
10 dbscan.core_sample_indices_
11
12 # 핵심 포인트 자체
13 dbscan.components_
```

Predict() 메서드는 없음

I 응용: 데이터 전처리(preprocessing)

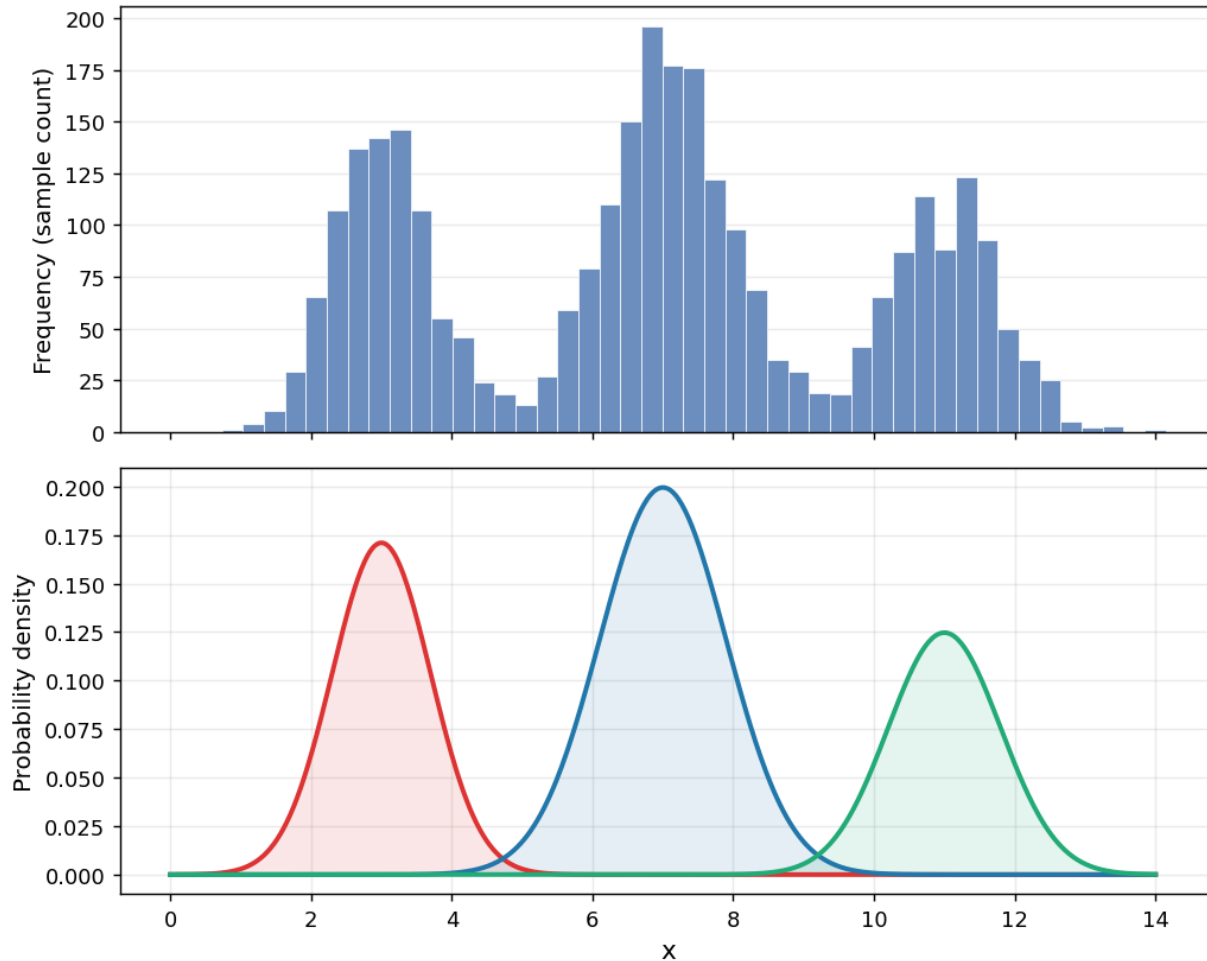
▶ 분류기 훈련을 위한 전처리 단계로 활용 가능

- 예: 레이블이 없는 상황, 준지도 학습(semi-supervised learning) 등

```
1 from sklearn.cluster import DBSCAN
2 from sklearn.ensemble import RandomForestClassifier
3
4 dbscan = DBSCAN(eps=0.05, min_samples=5)
5 dbscan.fit(X)
6
7 # 샘플이 속한 클러스터(-1: 이상치)
8 dbscan.labels_
9
10 # 핵심 포인트 인덱스
11 dbscan.core_sample_indices_
12
13 # 핵심 포인트 샘플(특징)
14 dbscan.components_
15
16 # 분류기 훈련 및 예측
17 clf = RandomForestClassifier(n_estimators=150, random_state=42)
18 clf.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])
19
20 clf.predict(X_new)
```

가우시안 혼합 모델(GMM, Gaussian Mixture Model)

▶ 데이터가 여러 개의 정규 분포로부터 섞여 생성되었다고 가정하는 확률 모델

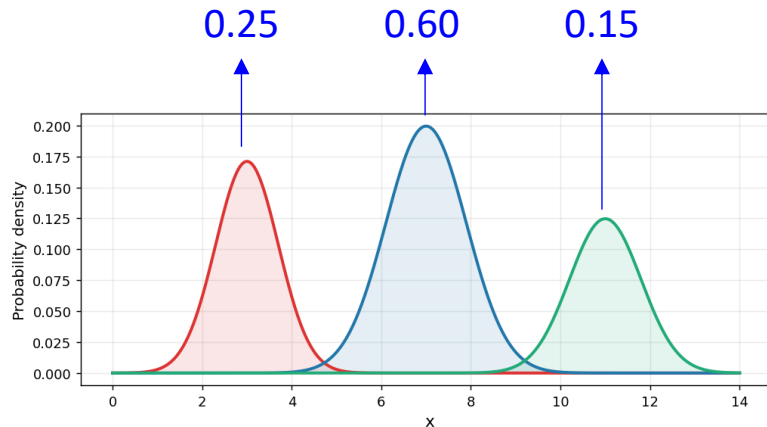


모델 훈련 개념

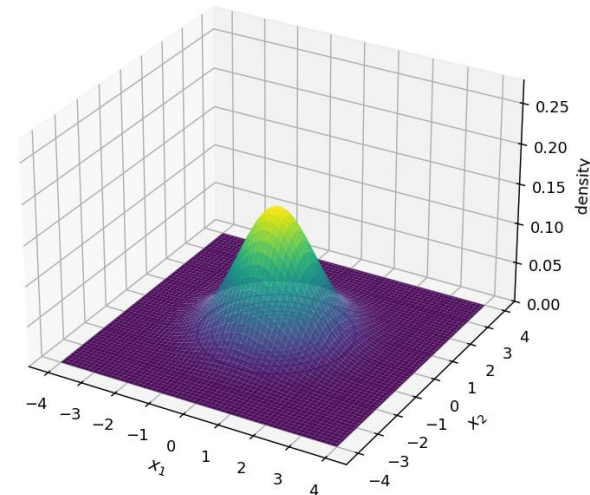
▶ 데이터셋 $\mathbf{x}$ 에 대해 확률 $p(\mathbf{x})$ 를 최대화하는 매개변수(π_k, μ_k, Σ_k)를 추정

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)$$

혼합 계수
(k 번째 분포가 전체에서 차지하는 비율)



다변량 정규분포
(multivariate normal distribution)

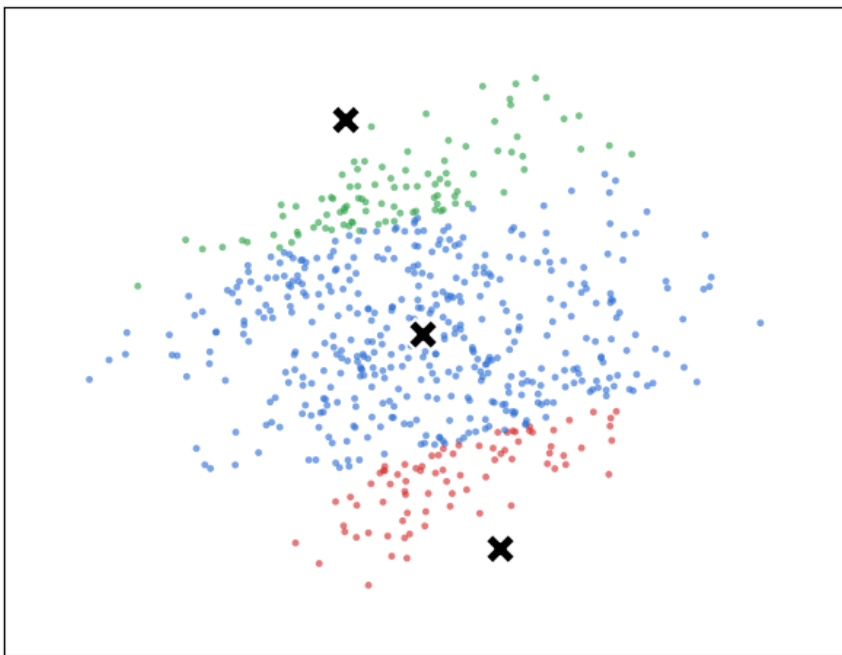


모델 훈련

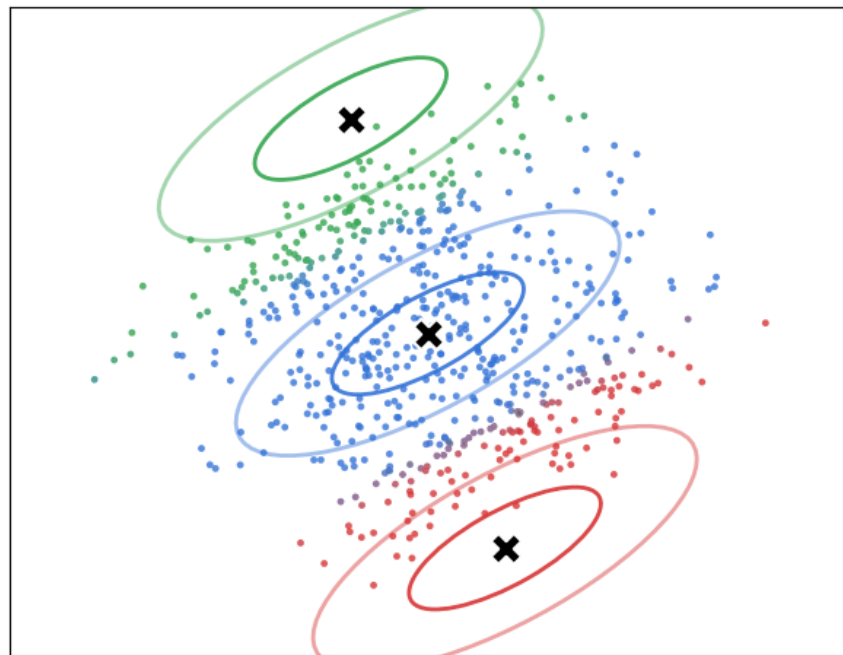
▶ 기댓값-최대화(EM, expectation-maximization) 알고리즘 활용

- **E-step**: 현재 파라미터에 기반해 각 클러스터에 속할 **확률** 예측(soft assignment)
- **M-step**: **확률**을 가중치로 사용해 각 클러스터의 파라미터를 업데이트

K-means (hard assignment)
iteration 0



GMM (soft assignment + contours)
iteration 0



■ 사이킷런 클래스: GaussianMixture

▶ 매개변수

- `n_components`: 가우시안 분포 갯수
- `n_init`: 다른 초기화로 EM 알고리즘 반복 횟수 → 가장 좋은 GMM 모델 선택

```
1 from sklearn.mixture import GaussianMixture
2
3 gmm = GaussianMixture(n_components=2, n_init=10, random_state=42)
4
5 gmm.fit(X)
6
7 gmm.predict(X)
```

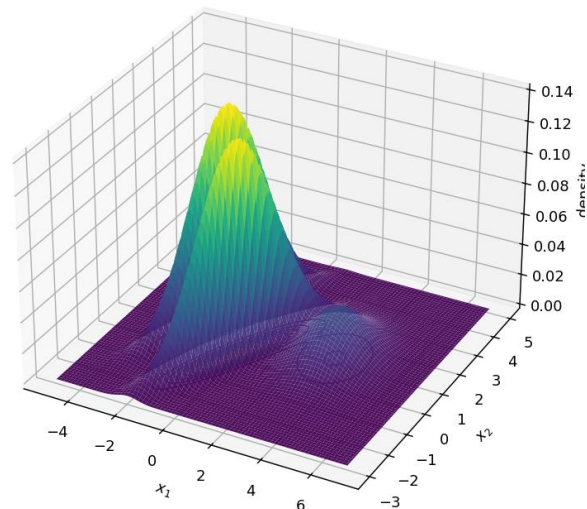
```
>>> gm.weights_
array([0.39025715, 0.40007391, 0.20966893])

>>> gm.means_
array([[ 0.05131611,  0.07521837],
       [-1.40763156,  1.42708225],
       [ 3.39893794,  1.05928897]])

>>> gm.covariances_
array([[[ 0.68799922,  0.79606357],
        [ 0.79606357,  1.21236106]],

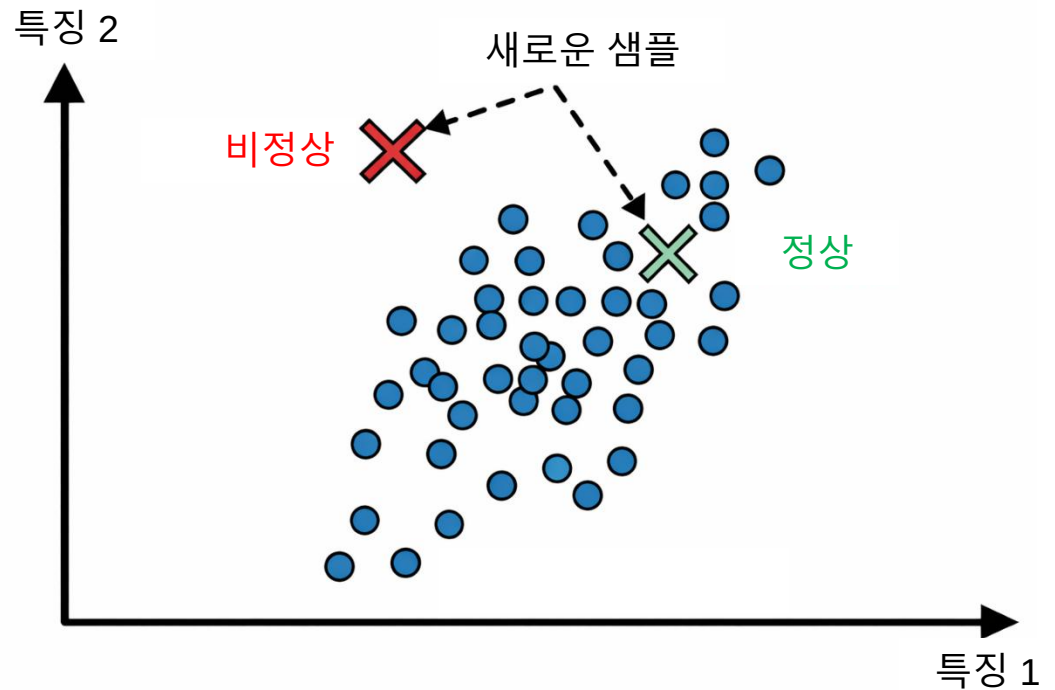
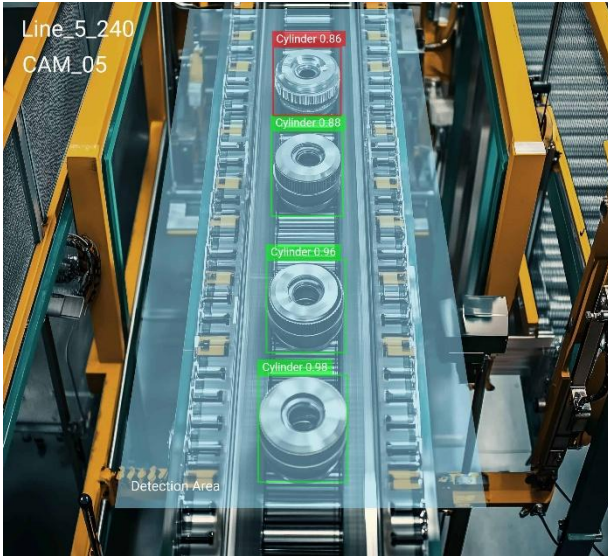
       [[ 0.63479409,  0.72970799],
        [ 0.72970799,  1.1610351 ]],

       [[ 1.14833585, -0.03256179],
        [-0.03256179,  0.95490931]]])
```



■ 응용: 이상치 탐지(Anomaly detection)

- ▶ 정상적인 데이터 패턴을 학습한 뒤, 비정상적인 데이터를 찾아내는 작업

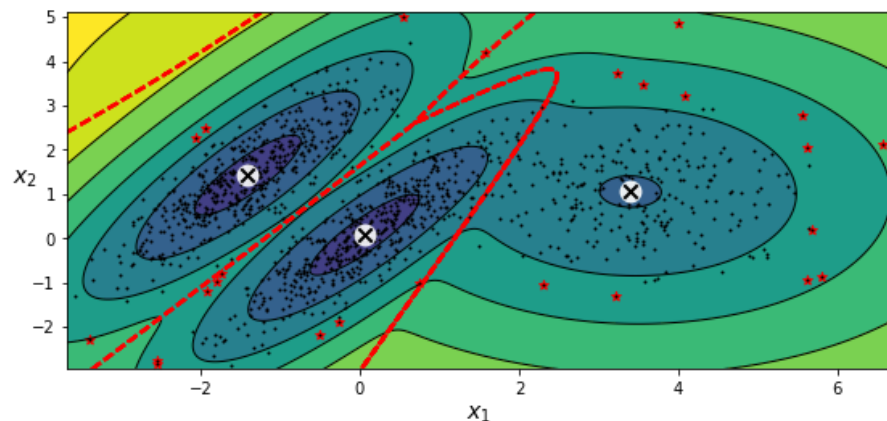
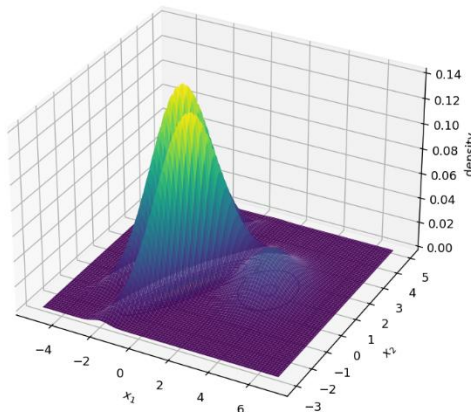


응용: 이상치 탐지(Anomaly detection)

▶ 특정 샘플의 확률 밀도가 낮다면 이상치로 간주



```
1 # 확률 밀도 함수의 로그(log)값 예측
2 log_dens = gm.score_samples(X)
3
4 # 임계값 설정 (하위 5%를 이상치로 판단)
5 threshold = np.percentile(log_dens, 5)
6 anomalies = X[log_dens < threshold]
```



▮ K-Means Clustering

- ▶ 중심점(centroid) 기반 군집(cluster) 알고리즘

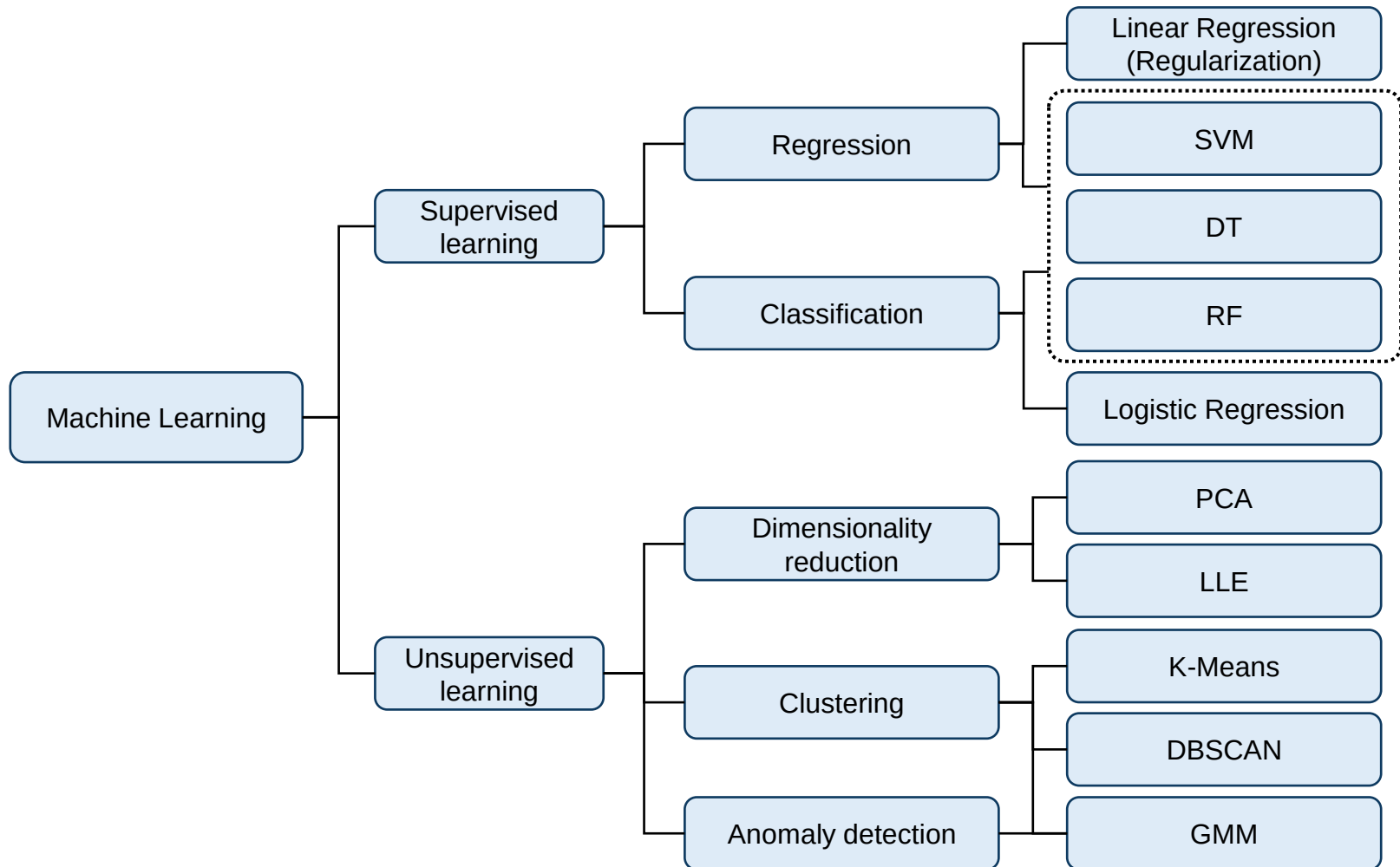
▮ DBSCAN

- ▶ 밀도(density) 기반 군집 알고리즘

▮ Gaussian Mixture Model (GMM)

- ▶ 데이터셋에 대한 확률 밀도 함수(PDF)를 추정

Wrap up



실 습

■ 실습 준비

- ▶ LMS에 업로드 된 파일 다운로드
- ▶ 프로젝트 폴더 열기
- ▶ 가상환경 설정 확인

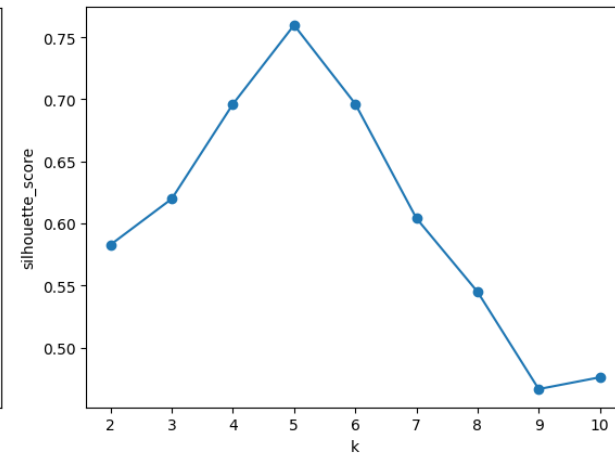
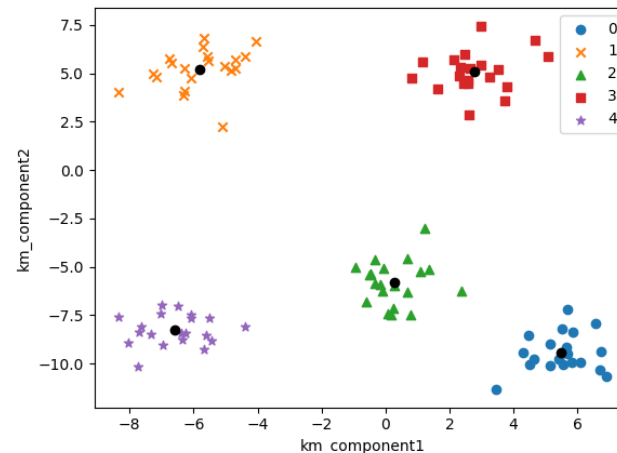
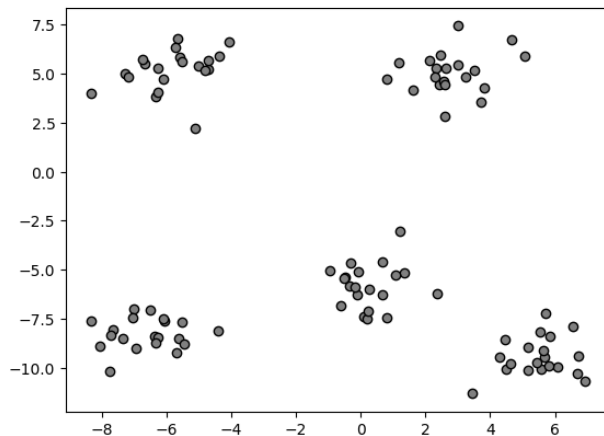
PCA

- ▶ 데이터셋에 PCA 적용
- ▶ PCA 적용 전 랜덤 포레스트 분류기 훈련
- ▶ PCA 적용 후 랜덤 포레스트 분류기 훈련
- ▶ 정확도 비교

실습 1. K-Means Clustering

K-Means Clustering

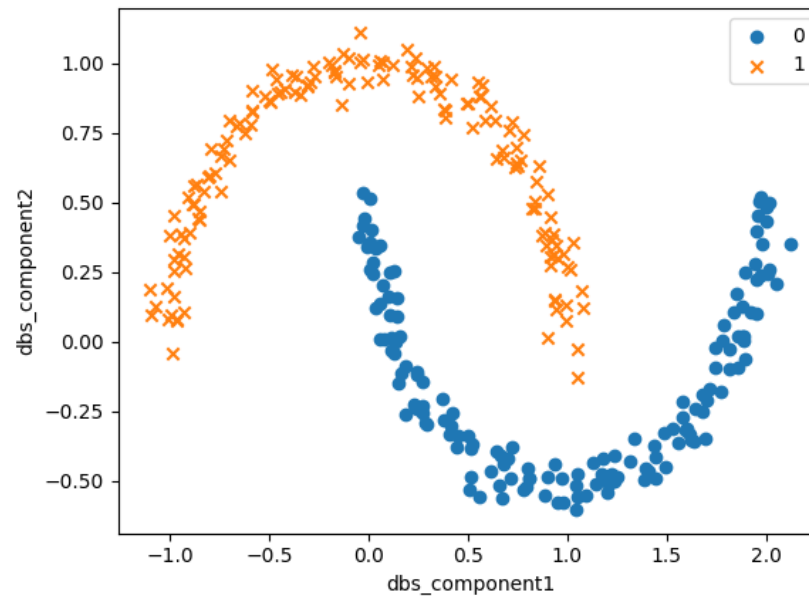
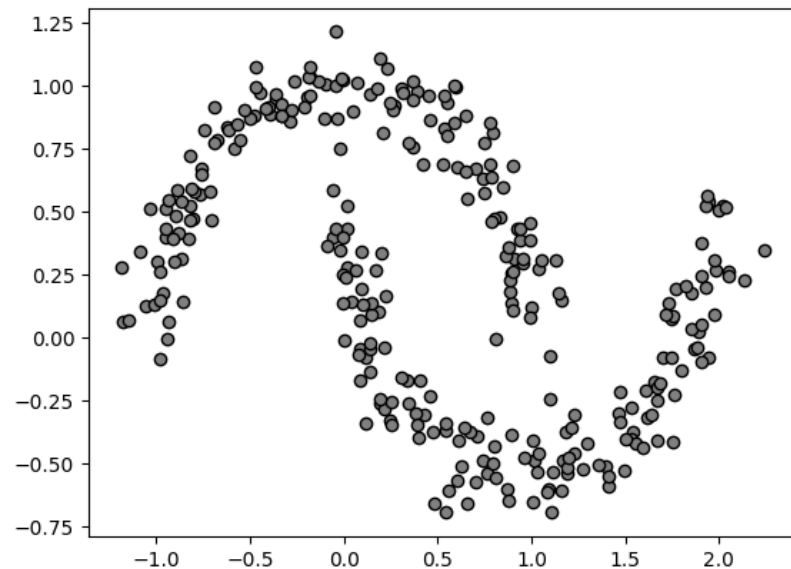
- ▶ 데이터 생성 및 시각화
- ▶ K-Means Clustering 알고리즘 훈련
- ▶ K-Means Clustering 결과 확인 및 시각화
- ▶ K 값 변화에 따른 실루엣 점수 확인 및 시각화



실습 2. DBSCAN

DBSCAN

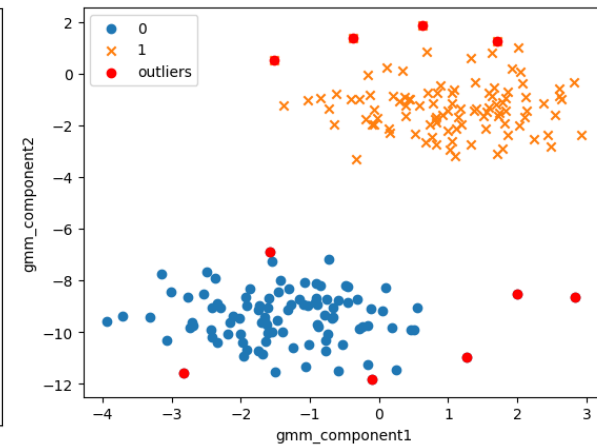
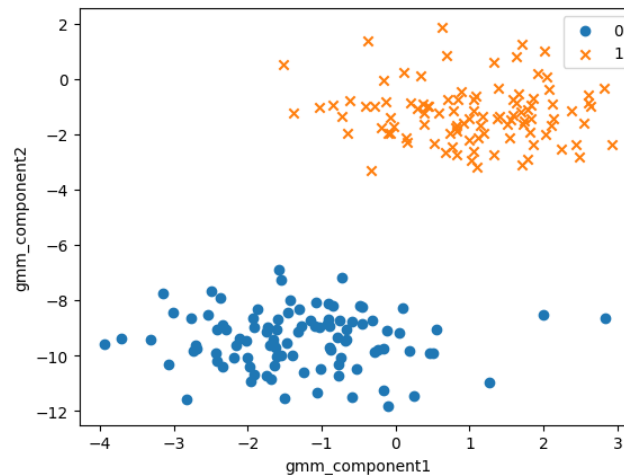
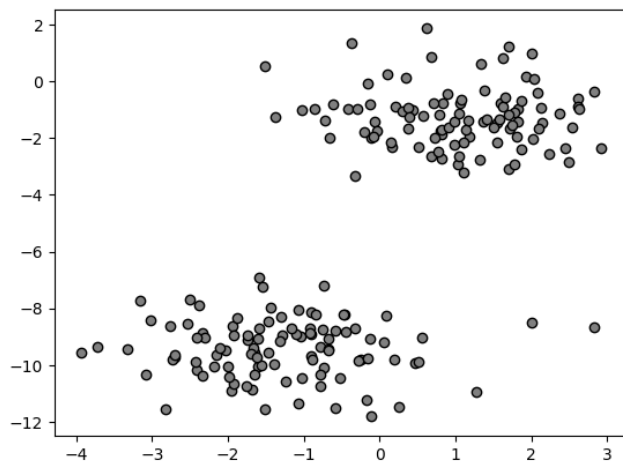
- ▶ 데이터 생성 및 시각화
- ▶ DBSCAN 알고리즘 훈련
- ▶ DBSCAN 결과 확인 및 시각화
- ▶ DBSCAN 응용: 라벨링 후 랜덤 포레스트 분류기 훈련 및 예측



실습 3. GMM

GMM

- ▶ 데이터 생성 및 시각화
- ▶ GMM 알고리즘 훈련
- ▶ GMM 결과 확인 및 시각화
- ▶ GMM 응용: 이상치 탐지



K-Means Clustering 응용

- ▶ 이미지 데이터에 대해 K-Means Clustering 훈련
- ▶ K 값에 따른 이미지 분할 결과 확인

